PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR 2025-2026 | FIFTH SEMESTER B.TECH

SOFTWARE ARCHITECTURE AND DESIGN (SAD Specification)

PROJECT: CAREERBRIDGE

Online Internship & Placement Preparation Portal

SUBMITTED BY: TEAM 4

Student Name	USN	Role Assigned	Deliverable
Deepika K	PES2UG24CS148	Team Lead & Architect	Architecture, ADRs, Component Diagram
Gunavathi M I	PES2UG24CS172	Frontend & UI/UX Lead	Design Overview, UX Design, Sequence Diagrams
Darshan R Mali	PES2UG24CS144	Backend & Database Architect	API Design, Data Stores, Security Architecture
Dhanush Rao	PES2UG24CS153	Full-Stack & QA/DevOps Lead	Test Strategy, Metrics, CI/CD Pipeline

Version: 1.0 (Draft for Review) | Date: 15-09-2026 | Status: Draft

Repository: <to be added by Team 4>

Software Architecture and Design Specification

Version: 1.0
Authors: Team 4 (CareerBridge)
Date: 15-09-2026
Status: Draft for Faculty Review

Revision History

Version	Date	Author	Change Summary
0.1	12-09-2026	Team 4	Initial draft structure and component identification
1.0	15-09-2026	Team 4	Complete SAD with component diagram, 2 sequence diagrams, API design and STRIDE threat model

Approvals

Role	Name	Department / Contact	Sign / Date
Team Lead	Dhanush Rao	PES2UG24CS153 (CSE)	
Course Coordinator	Software Engineering Faculty	Dept of CSE, PES University	

1. Introduction

1.1 Purpose

This Software Architecture and Design (SAD) document specifies the architecture, component structure, design patterns, API contracts, and security architecture of CareerBridge, an online internship and placement preparation portal for students. It serves as the authoritative technical reference for developers, quality assurance engineers, and assessment evaluators, providing complete traceability from architectural decisions to the functional and non-functional requirements defined in the Software Requirements Specification (SRS v1.0).

1.2 Scope

This document covers the complete software architecture and detailed design of CareerBridge, including:

- Component-based layered architecture of the React single-page application
- Node.js/Express REST API and MongoDB data persistence layer
- A supporting Python (Flask) microservice for the aptitude test engine and progress analytics
- UML component diagram illustrating all system modules and their inter-dependencies
- UML sequence diagrams modeling two core application workflows
- RESTful API design for internship listings, applications, and aptitude tests
- STRIDE-based threat modeling and security architecture
- Technology stack rationale and architectural decision records

Out of scope: third-party job-portal internals, browser rendering engine internals, and campus placement-cell administrative workflows outside the portal.

1.3 Audience

- Developers: Reference for implementing and extending CareerBridge components
- QA Engineers: Basis for deriving integration and system test cases from architectural contracts
- Security Auditors: STRIDE threat model and security control documentation
- Course Evaluators: Assessment of architectural design quality and IEEE 42010 compliance
- Maintenance Engineers: Component dependency map for impact analysis during future modifications

1.4 Definitions

Term	Definition
SAD	Software Architecture and Design Specification
SRS	Software Requirements Specification
STP	Software Test Plan
SPA	Single Page Application
JWT	JSON Web Token (used for stateless session authentication)
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
ADR	Architecture Decision Record
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
RTM	Requirements Traceability Matrix

TLS	Transport Layer Security
ATS	Applicant Tracking System (resume/application screening)

2. Document Overview

2.1 How to Use This Document

This document is organized into three primary sections:

- Section 3 (Architecture): Provides the high-level system decomposition, component relationships, technology stack decisions, and security architecture. Start here for understanding the overall system structure.
- Section 4 (Design): Contains detailed design specifications including sequence diagrams for core workflows, API contracts, error handling strategies, and UX design principles. Reference this section for implementation-level guidance.
- Section 5 (Appendices): Contains the glossary, normative references, and tooling documentation.

All UML diagrams conform to UML 2.5 notation standards. Architecture decisions follow the Architecture Decision Record (ADR) format where applicable.

2.2 Related Documents

Document	Version	Notes
Software Requirements Specification (SRS)	1.0	Maintained by Team 4
Software Test Plan (STP)	1.0	Companion document to this SAD
Requirements Traceability Matrix (RTM)	1.0	Included in SRS
Database Schema (DDL/Schema definition)	1.0	database/schema.js (Mongoose)
CI/CD Workflow Definition	1.0	.github/workflows/ci.yml

3. Architecture

3.1 Goals and Constraints

Architectural Goals:

- Modularity: Independent development and testing of each functional module (Listings, Applications, Aptitude Tests, Analytics)
- Accessibility: A responsive interface usable by students on both desktop and mobile devices
- Security: Defense-in-depth strategy with JWT authentication, input sanitization, and encrypted transport
- Performance: Internship search and filtering results returned within 2 seconds under normal load
- Maintainability: Clear component boundaries between the React frontend, Node/Express API, and the Python analytics microservice
- Availability: Target 99.5% uptime during placement season with graceful degradation of non-critical features

Architectural Constraints:

- Must operate as a client-side SPA served independently of the backend API (decoupled frontend/backend)
- Must integrate a MongoDB document store using the Mongoose ODM
- The aptitude-test scoring and analytics engine must be implemented in Python to leverage pandas/scikit-learn, and exposed to the Node backend as an internal microservice
- Must pass automated CI/CD validation (build, test, lint) on every push and pull request via GitHub Actions
- Technology stack must align with the course-approved options: Python / JavaScript (MERN Stack)

3.2 Stakeholders and Concerns

Stakeholder	Primary Concerns
Team Lead (Deepika)	System reliability, architecture coherence, deployment simplicity, team coordination
Frontend Developer (Gunavathi)	UI responsiveness, component reusability, accessibility, visual consistency
Backend Developer (Darshan)	Data integrity, schema design, API correctness, query performance
QA/DevOps Lead (Dhanush)	Test coverage, CI/CD pipeline reliability, build reproducibility, defect tracking
Course Evaluators	IEEE compliance, documentation quality, functional completeness, collaboration evidence
End Users (Students)	Ease of discovering internships, fair aptitude assessment, visibility into readiness and progress
Placement Cell / Recruiters	Accurate applicant data, reliable listing management, exportable reports

3.3 Component (UML) Diagram

The following UML Component Diagram illustrates the high-level decomposition of CareerBridge into its constituent modules, their provided/required interfaces, and inter-component dependencies:

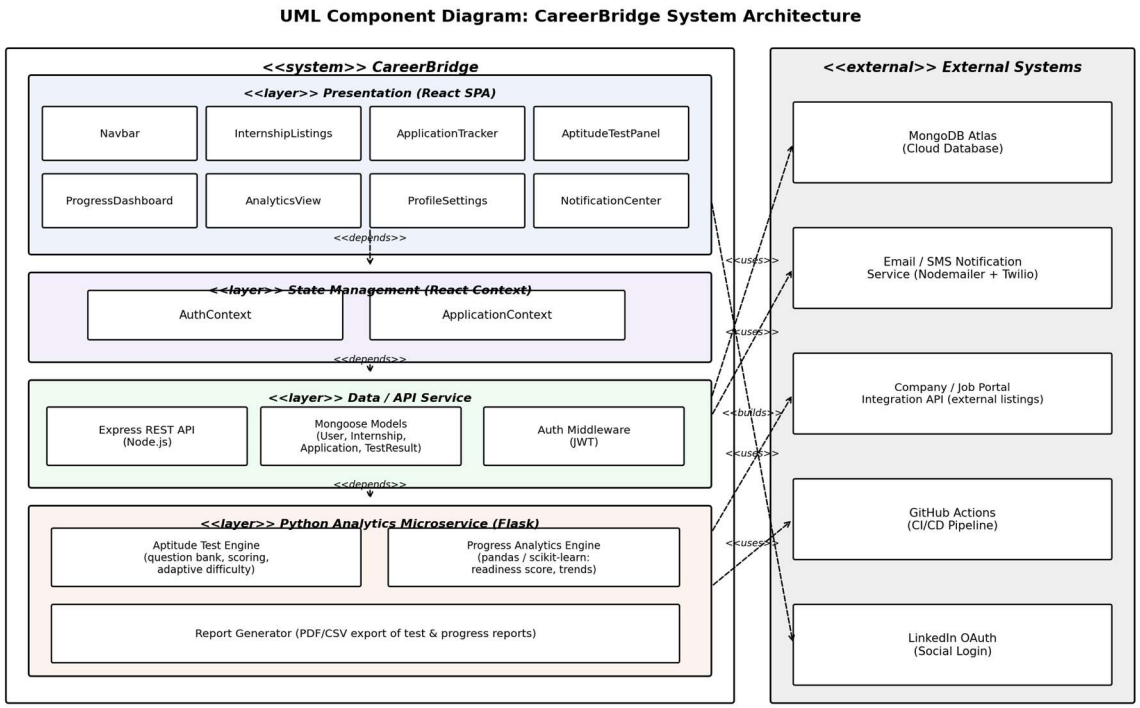


Figure 3.1: UML Component Diagram – CareerBridge System Architecture

The system is decomposed into the following primary components:

- Presentation Layer: Navbar, InternshipListings, ApplicationTracker, AptitudeTestPanel, ProgressDashboard, AnalyticsView, ProfileSettings, NotificationCenter
- State Management Layer: AuthContext, ApplicationContext
- Data / API Service Layer: Express REST API, Mongoose Models, JWT Auth Middleware
- Python Analytics Microservice: Aptitude Test Engine, Progress Analytics Engine, Report Generator
- External Systems: MongoDB Atlas, Email/SMS Notification Service, Company/Job Portal Integration API, GitHub Actions CI/CD, LinkedIn OAuth

3.4 Component Descriptions

Component	Module Path	Responsibility	Dependencies
App Shell	src/App.jsx	Root application container; manages routing and global layout	AuthContext, Navbar
Navbar	src/components/layout/Navbar.jsx	Top navigation with profile display and notification bell	AuthContext
InternshipListings	src/components/listings/InternshipListings.jsx	Searchable, filterable list of internship postings	ApplicationContext
ApplicationTracker	src/components/applications/ApplicationTracker.jsx	Displays status of a student's submitted applications	ApplicationContext
AptitudeTestPanel	src/components/tests/AptitudeTestPanel.jsx	Timed aptitude test UI: question rendering, timer, submission	ApplicationContext
ProgressDashboard	src/components/dashboard/ProgressDashboard.jsx	Readiness score, test history and trend charts	ApplicationContext

AnalyticsView	src/components/analytics/AnalyticsView.jsx	Aggregate analytics: strengths/weaknesses by topic	ApplicationContext
ProfileSettings	src/components/settings/ProfileSettings.jsx	Resume upload, skills, and preference configuration	AuthContext
NotificationCenter	src/components/notifications/NotificationCenter.jsx	In-app and email/SMS notification preferences and history	AuthContext
AuthContext	src/context/AuthContext.jsx	React Context managing authentication state and JWT session	Express API
ApplicationContext	src/context/ApplicationContext.jsx	React Context managing internships, applications, and test data	Express API, AuthContext
Express REST API	server/routes/*.js	Node.js API gateway exposing REST endpoints, request validation	Mongoose Models, JWT Middleware
Mongoose Models	server/models/*.js	Schema definitions for User, Internship, Application, TestResult	MongoDB Atlas
Aptitude Test Engine	analytics_service/test_engine.py	Question bank management, scoring, adaptive difficulty (Flask)	MongoDB Atlas (via API)
Progress Analytics Engine	analytics_service/analytics.py	Readiness score and trend computation using pandas	Aptitude Test Engine
Report Generator	analytics_service/reports.py	Exports PDF/CSV test and progress reports	Progress Analytics Engine

3.5 Chosen Architecture Pattern and Rationale

Selected Pattern: Layered Component Architecture with a Polyglot Analytics Microservice

Rationale:

Criterion	Layered + Microservice (Chosen)	Full Microservices	Monolithic MVC
Separation of Concerns	Strong (component + service boundary)	Very Strong (full isolation)	Moderate
Development Complexity	Low-Medium	High (orchestration overhead)	Low
Team Parallelism	High (4 independent modules)	Very High	Low (merge conflicts)
Deployment Simplicity	Medium (2 services)	Low (many services)	High
Fit for Python analytics needs	Excellent (dedicated Flask service)	Excellent but overkill	Poor (JS-only)
Academic Evaluation Fit	Good (npm run dev + flask run)	Poor (needs orchestration)	Good

Decision: A layered component architecture was chosen for the React frontend and Node/Express backend because it provides strong separation of concerns while keeping deployment simple. A single focused Python (Flask) microservice was introduced specifically for the aptitude test engine and progress analytics, since these features benefit from Python's data-science ecosystem (pandas, scikit-learn) that is not natively available in the Node.js runtime. A full microservices split was rejected as unnecessary complexity for the project scope; a pure monolithic JavaScript stack was rejected because it would have made the analytics workloads harder to implement and maintain.

3.6 Technology Stack and Data Stores

Layer	Technology	Version	Justification
UI Framework	React	18.x	Component-based SPA with a mature ecosystem and hooks-based state management
Build Toolchain	Vite	5.x	Fast dev server and optimized production bundling

State Management	React Context API	Built-in	Sufficient for project scope; avoids external dependency overhead
Backend API	Node.js + Express	20.x / 4.x	Lightweight REST API layer, large middleware ecosystem
Database	MongoDB (Atlas)	7.x	Flexible document schema suited to varied internship/application data
ODM	Mongoose	8.x	Schema validation and query building for MongoDB
Analytics Microservice	Python + Flask	3.x / 3.0	Exposes aptitude scoring and analytics via internal REST endpoints
Data Analysis	pandas, scikit-learn	Latest	Readiness scoring, trend analysis, and adaptive difficulty logic
Authentication	JSON Web Tokens (JWT)	—	Stateless session management across frontend and API
Unit Testing	Jest (JS) / PyTest (Python)	Latest	Test coverage for both the Node API and Python microservice
CI/CD	GitHub Actions	Latest	Automated build, lint, and test pipeline on push/PR
Transport Security	TLS 1.2+	—	Enforced for all client-server and inter-service communication

Data Store Architecture (MongoDB Collections):

- users – student profile, credentials (hashed), resume metadata
- internships – postings sourced internally or via integration API
- applications – links a user to an internship with status (applied, shortlisted, rejected, offered)
- testResults – aptitude test attempts, scores, per-topic breakdown
- progressSnapshots – periodic readiness-score snapshots computed by the analytics engine

3.7 Risks and Mitigations

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
RISK-01	MongoDB Atlas free-tier connection limits reached during peak usage	Medium	Medium	Connection pooling in Mongoose; upgrade tier before placement season
RISK-02	Python analytics microservice becomes a single point of failure	Low	High	API gracefully degrades: cached last-known readiness score shown if service is unreachable
RISK-03	Aptitude test question bank duplication across attempts reduces validity	Medium	Medium	Randomized question selection with per-student attempt history exclusion
RISK-04	Inconsistent internship data from external integration API	Medium	Low	Schema validation and sanitization layer before persistence
RISK-05	CI/CD pipeline failure blocking Pull Request merges	Low	Medium	Separate build/test jobs with independent failure isolation
RISK-06	Resume/file upload used to inject malicious content	Low	High	File-type whitelisting, size limits, and virus-scan hook before storage

3.8 Traceability to Requirements

The following table traces architectural components to representative functional requirements defined in SRS v1.0:

Requirement ID	Requirement Description	Architectural Component(s)
CB-F-001	Student registration and login	AuthContext, Express API, Mongoose (users)
CB-F-002	Browse and filter internship listings	InternshipListings, Express API
CB-F-003	Apply to an internship	ApplicationTracker, ApplicationContext, Express API

CB-F-004	Attempt a timed aptitude test	AptitudeTestPanel, Aptitude Test Engine (Flask)
CB-F-005	Adaptive difficulty based on performance	Aptitude Test Engine
CB-F-006	View readiness score and progress trends	ProgressDashboard, Progress Analytics Engine
CB-F-007	Export progress/test report	Report Generator
CB-F-008	Receive email/SMS notifications on application status change	NotificationCenter, Notification Service

3.9 Security Architecture

Threat Modeling (STRIDE Analysis):

STRIDE Category	Threat Description	Countermeasure	Component
Spoofing	Unauthorized user impersonating a legitimate student	JWT-based session tokens with password hashing (bcrypt)	AuthContext, Express API
Tampering	Malicious modification of application or test-score data in transit	TLS enforced on all API calls; server-side input validation	Express API
Repudiation	Student denying they submitted a particular test attempt	Immutable testResults log with timestamp and user ID	Aptitude Test Engine
Information Disclosure	Unauthorized access to another student's resume or application data	Ownership checks on every query; field-level access control	Express API, Mongoose
Denial of Service	Excessive test-submission requests overwhelming the analytics service	Rate limiting on submission endpoints; request queueing	Express API, Aptitude Test Engine
Elevation of Privilege	Student attempting to access placement-cell admin functions	Role-based access control enforced server-side, not just hidden in UI	AuthContext, Express API

Security Controls Summary:

- Authentication: JWT-based session management with bcrypt password hashing
- Authorization: Ownership and role checks enforced in Express middleware, not only client-side
- Input Sanitization: Server-side validation plus React JSX auto-escaping on the client
- Transport Security: TLS 1.2+ enforced on all client-server and service-to-service communication
- File Upload Safety: Type/size restrictions and scanning on resume uploads
- Dependency Security: Automated npm audit and pip-audit in the CI/CD pipeline

4. Design

4.1 Design Overview

CareerBridge is designed as a modular, component-based single-page application with a companion analytics microservice, following separation of concerns at every layer:

- Presentation Layer: React functional components with hooks-based lifecycle management, each module (Listings, Applications, Tests, Dashboard) independently encapsulated
- State Management Layer: Two React Context providers (AuthContext and ApplicationContext) serve as centralized state containers
- Data Access Layer: The Express REST API routes all CRUD operations to MongoDB via Mongoose, and delegates aptitude-test scoring and analytics to the Python Flask microservice over an internal HTTP call

This layered design enables each team member to develop and test their assigned components independently, with well-defined interfaces at layer boundaries.

4.2 UML Sequence Diagrams

4.2.1 Sequence Diagram 1: Student Registration & Internship Application Workflow

This sequence diagram models the end-to-end flow from student registration through browsing and applying to an internship listing:

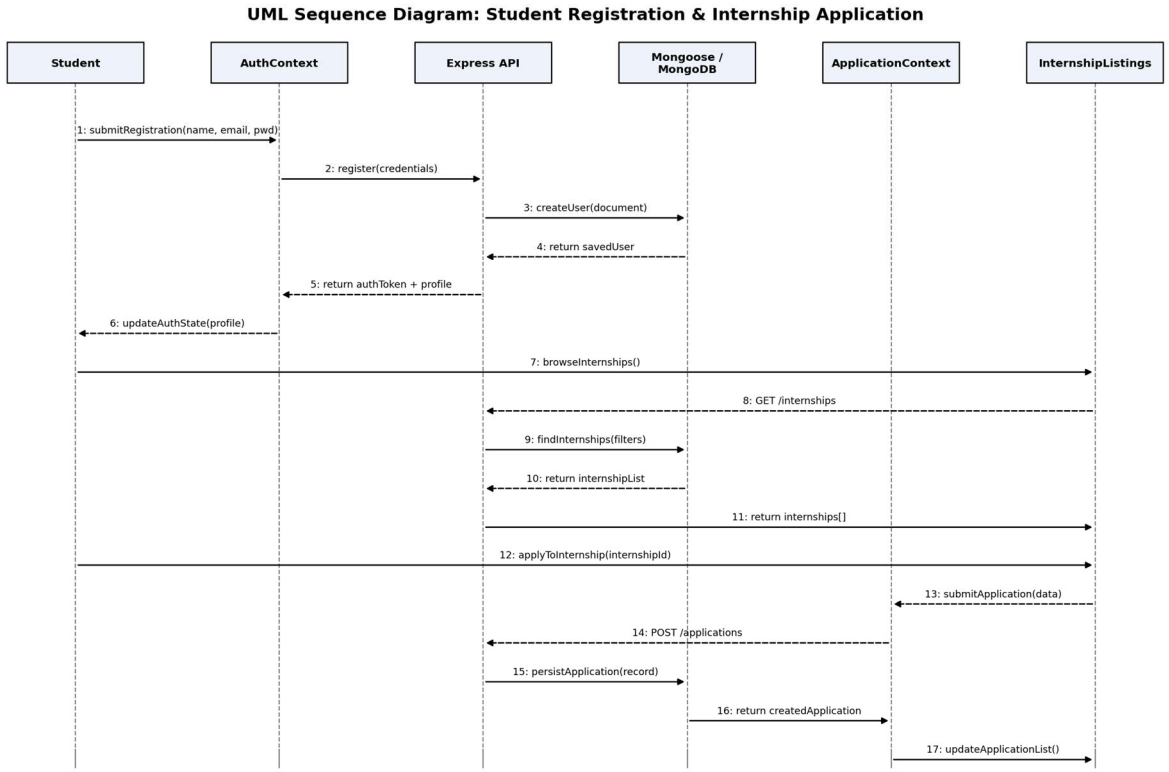


Figure 4.1: UML Sequence Diagram – Registration and Internship Application

Flow Description:

- Student submits registration details; AuthContext dispatches the request to the Express API

- Express API creates the user document in MongoDB and returns an auth token and profile
- AuthContext updates client-side auth state; the student browses internship listings
- InternshipListings requests data from the API, which queries MongoDB and returns matching internships
- Student applies to a chosen internship; ApplicationContext submits the application
- The API persists the application record and the application list is refreshed on screen

4.2.2 Sequence Diagram 2: Aptitude Test Attempt & Progress Analytics Workflow

This sequence diagram models a student attempting an aptitude test and reviewing computed progress analytics:

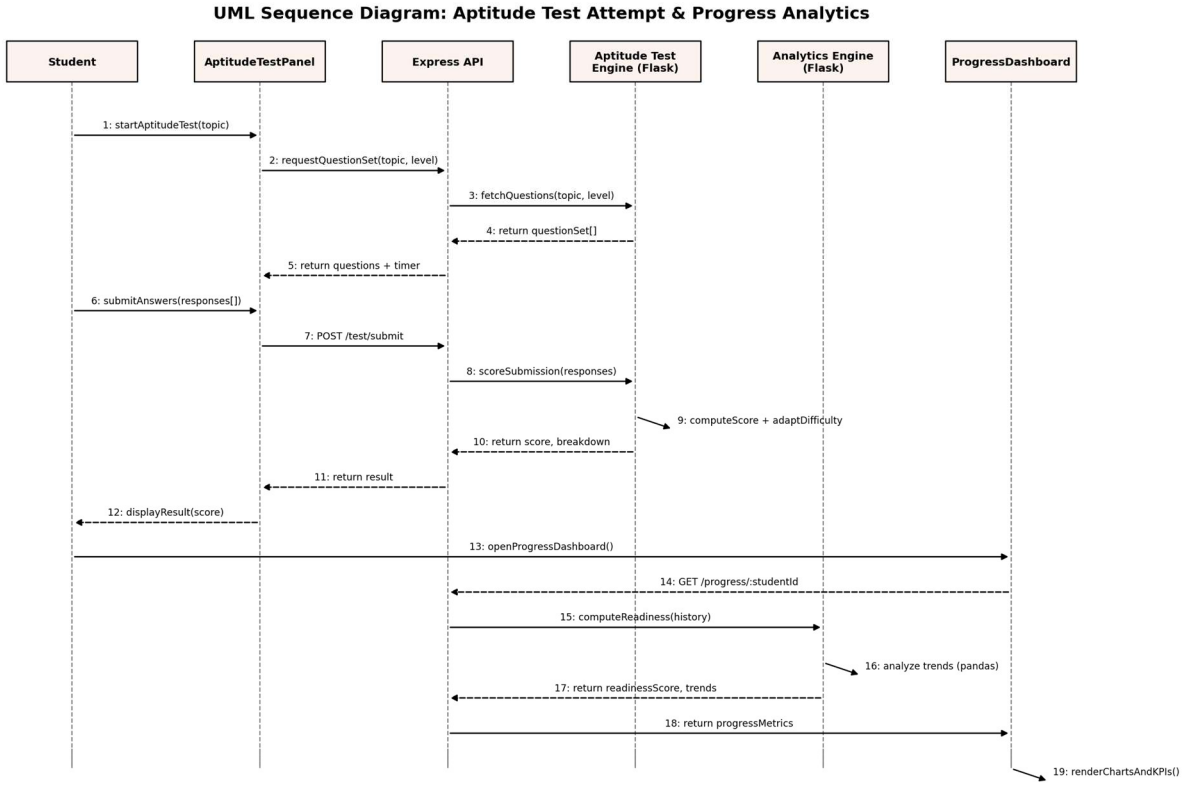


Figure 4.2: UML Sequence Diagram – Aptitude Test Attempt and Progress Analytics

Flow Description:

- Student starts an aptitude test for a chosen topic; the API requests a question set from the Aptitude Test Engine
- Student submits answers, which are scored by the Aptitude Test Engine, including adaptive difficulty adjustment for future attempts
- Student opens the Progress Dashboard; the API requests readiness computation from the Analytics Engine
- The Analytics Engine analyzes historical attempts with pandas to compute a readiness score and topic-level trends
- The dashboard renders charts and KPI cards summarizing the student's placement readiness

4.3 API Design

CareerBridge exposes a RESTful API from the Express backend; the analytics microservice exposes a small internal API consumed only by the Express layer (not the browser).

4.3.1 Internships API

Endpoint	Method	Description
/api/internships	GET	Retrieve internship listings with optional filters (location, domain, stipend)
/api/internships	POST	Create a new internship listing (placement-cell role only)
/api/internships/:id	GET	Retrieve a single internship's details
/api/applications	POST	Submit a new application to an internship
/api/applications/:studentId	GET	Retrieve a student's application history and statuses

Request Schema (POST /api/applications):

```
{
  "internshipId": "ObjectId (required)",
  "studentId": "ObjectId (required)",
  "resumeUrl": "string (required)",
  "coverNote": "string (optional, max 1000 chars)"
}
```

Error Responses:

- 401 Unauthorized – Missing or invalid authentication token
- 403 Forbidden – Student attempting to reapply after rejection lockout period
- 422 Unprocessable Entity – Validation failure (missing required fields, invalid enum value)

4.3.2 Aptitude Test API (internal, via Express → Flask)

Endpoint	Method	Description
/api/test/questions	GET	Fetch a question set for a given topic and difficulty level
/api/test/submit	POST	Submit responses for scoring; returns score and per-topic breakdown
/api/progress/:studentId	GET	Retrieve computed readiness score and trend data
/api/reports/:studentId	GET	Generate and download a PDF/CSV progress report

Response Schema (200 OK, POST /api/test/submit):

```
{
  "score": "integer",
  "totalQuestions": "integer",
  "breakdown": "{ topic: string -> correctCount: integer }",
  "nextDifficultyLevel": "string (enum: easy | medium | hard)"
}
```

4.4 Error Handling, Logging and Monitoring

Error Handling Strategy:

- All API calls are wrapped in try-catch blocks with standardized error response parsing
- If the Python analytics microservice is unreachable, the API falls back to the last cached readiness score with a user-visible notice
- Form validation errors are displayed inline with field-level error messages
- Unrecoverable client errors are caught by a React Error Boundary that renders a graceful fallback UI

Logging and Monitoring:

- Application status changes and test submissions are logged with timestamp, user ID, and action type

- No sensitive information (passwords, tokens) is written to any log
- CI/CD pipeline provides automated build health monitoring on every push
- API response time and error-rate metrics are captured via console/server logs for local monitoring

4.5 UX Design

Design Principles:

- Clarity: Application statuses and test results are shown with clear, color-coded indicators
- Progressive Disclosure: Advanced filters and settings are tucked into secondary panels to keep the primary flow simple
- Accessibility: Readable font sizes, sufficient color contrast, and keyboard-navigable forms
- Responsive Layout: Usable from mobile browsers (360px) up through desktop widescreen layouts

Visual Design System:

- Color Palette: Clean light theme with an accent color for calls-to-action and status badges
- Typography: System font stack for readability across platforms
- Spacing: 8px grid system for consistent padding, margins, and component sizing

4.6 Open Issues and Next Steps

Issue	Status	Next Step
Resume parsing to auto-fill profile fields	Not started	Evaluate a resume-parsing library or lightweight NLP model
Real-time notifications (WebSocket push)	Deferred	Evaluate Socket.IO for live application-status updates
Recruiter-facing dashboard for shortlisting	Not started	Design a separate role-gated view for placement-cell staff
Mobile app wrapper	Not started	Evaluate a PWA wrapper around the existing React SPA

5. Appendices

5.1 Glossary

Term	Definition
Internship Listing	A posting describing an internship opportunity, including role, domain, stipend, and eligibility
Aptitude Test	A timed, topic-based assessment used to gauge a student's placement readiness
Readiness Score	A computed metric summarizing a student's aggregate preparedness across topics and attempts
Adaptive Difficulty	Adjustment of subsequent question difficulty based on a student's prior performance
ATS	Applicant Tracking System style status pipeline (applied → shortlisted → offered/rejected)

5.2 References

Reference	Description
IEEE 42010:2011	Systems and Software Engineering – Architecture Description
OWASP Top 10 (2021)	Open Web Application Security Project – Top 10 Web Application Security Risks
React Documentation	Official React 18 documentation (react.dev)
Express.js Documentation	Official Express.js framework documentation (expressjs.com)
MongoDB / Mongoose Documentation	Official MongoDB and Mongoose ODM documentation
Flask Documentation	Official Flask microframework documentation (flask.palletsprojects.com)

5.3 Tools

Tool	Purpose
Visual Studio Code	Primary development IDE
Git / GitHub	Version control and team collaboration
GitHub Actions	CI/CD pipeline automation
Postman	API testing for Express and Flask endpoints
Jest / PyTest	Unit testing frameworks for JS and Python code
Python (Matplotlib)	UML diagram generation for documentation